Assignment 1 - PS07: Emergency Route Planning using Greedy Best First Search
Group Id: G087

1. Problem Formulation
The disaster management control center needs a problem-solving agent that finds an emergency route from a start location to a goal location during flooding in Chennai. The city map is represented as a graph where each node is a location and each edge is a reachable road segment. The agent uses Greedy Best First Search, where f(n) = h(n), so the next expanded node is always the frontier node with the smallest estimated distance to the goal.

Initial State: the user-provided start node.
Goal Test: current node equals the goal node.
Actions: move from the current location to any connected neighboring location.
Path Cost: sum of edge costs along the returned route. If no edge cost is supplied in the input file, cost 1 is used.

2. PEAS Description
Performance Measure: reach the affected area quickly, produce a valid path, minimize heuristic distance at every choice, report the visited order, path, cost, measured runtime, and measured memory.
Environment: Chennai road network during heavy rain, including possible flooded or blocked roads represented by missing or high-cost edges.
Actuators: route selection decisions, expansion of next city node, and output of route guidance to rescue teams.
Sensors: input graph, heuristic table, start node, goal node, and edge-cost information.

3. Environmental Characteristics
The environment is partially observable because the program only knows the graph and heuristic values provided in the input. It is deterministic for the submitted implementation because each action leads to the listed neighbor with a known cost. It is sequential because each route decision affects the following state. It is static during one search run because the graph is not changed while the algorithm executes. It is discrete because locations and roads are represented as graph nodes and edges. It is single-agent from the search perspective because only one rescue route is planned by this agent.

4. Heuristic Function
The heuristic h(n) estimates how close a node n is to the goal. In this assignment's sample cases, lower heuristic values indicate locations that appear closer to the affected area. GBFS ignores accumulated path cost while choosing the next node; it expands the frontier node with the smallest h(n). This makes the search fast and goal-directed, but it does not guarantee the minimum-cost path for all weighted graphs.

5. Data Structures and Algorithm
The graph is stored as an adjacency list: node -> list of (neighbor, cost). Heuristic values are stored in a dictionary for O(1) lookup. The frontier is a bounded min-priority queue implemented using heapq. The queue explicitly raises an error if insertion is attempted when full or deletion is attempted when empty, satisfying the data-structure error-handling requirement. A visited set prevents repeated expansion of the same node. Each priority queue entry stores heuristic value, insertion order, current node, path so far, and accumulated cost.

Algorithm steps:
1. Read graph, heuristics, start node, and goal node from the input file.
2. Insert the start node in the priority queue with priority h(start).
3. Repeatedly remove the node with the lowest heuristic value.
4. If the node is the goal, print expansion order, path, cost, measured time, and measured memory.
5. Otherwise, insert all unvisited neighbors with priority h(neighbor).
6. If the frontier becomes empty, report that no route exists.

6. Complexity Reporting
The program prints measured runtime using time.perf_counter() and measured peak memory using tracemalloc for the actual input run, as required by the assignment instruction not to calculate these theoretically in the implementation. For design understanding, if V is the number of locations and E is the number of roads, the priority queue operations make the expected algorithmic time O(E log E) in the worst case and the auxiliary space O(E) for the frontier plus visited/path information.

7. Alternate Modeling Approach
An alternate model is to represent the problem as a weighted shortest-path search using A* with f(n) = g(n) + h(n), where g(n) is the cost from the start and h(n) is the estimated remaining cost. A* generally has higher bookkeeping cost than GBFS because it must track and compare accumulated path costs, but it can find an optimal path when the heuristic is admissible and consistent. GBFS is simpler and often faster for emergency guidance when a quick feasible route is more important than guaranteed optimality, but it may choose a longer route if the heuristic is misleading.

8. Sample Result
For Test Case 1, start A and goal G, the program expands:
A -> C -> F -> G
The returned path is:
A -> C -> F -> G
The total cost is 3 when every road segment has unit cost.